Estructuras de Datos

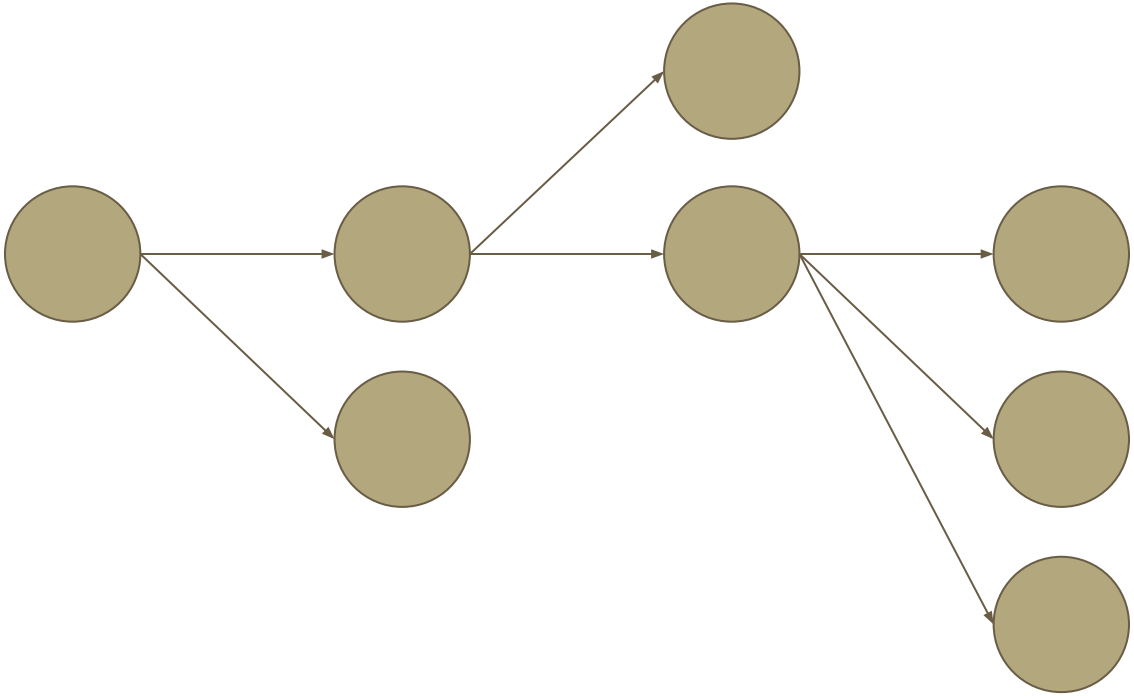
2026
Segundo Cuatrimestre

REPASO

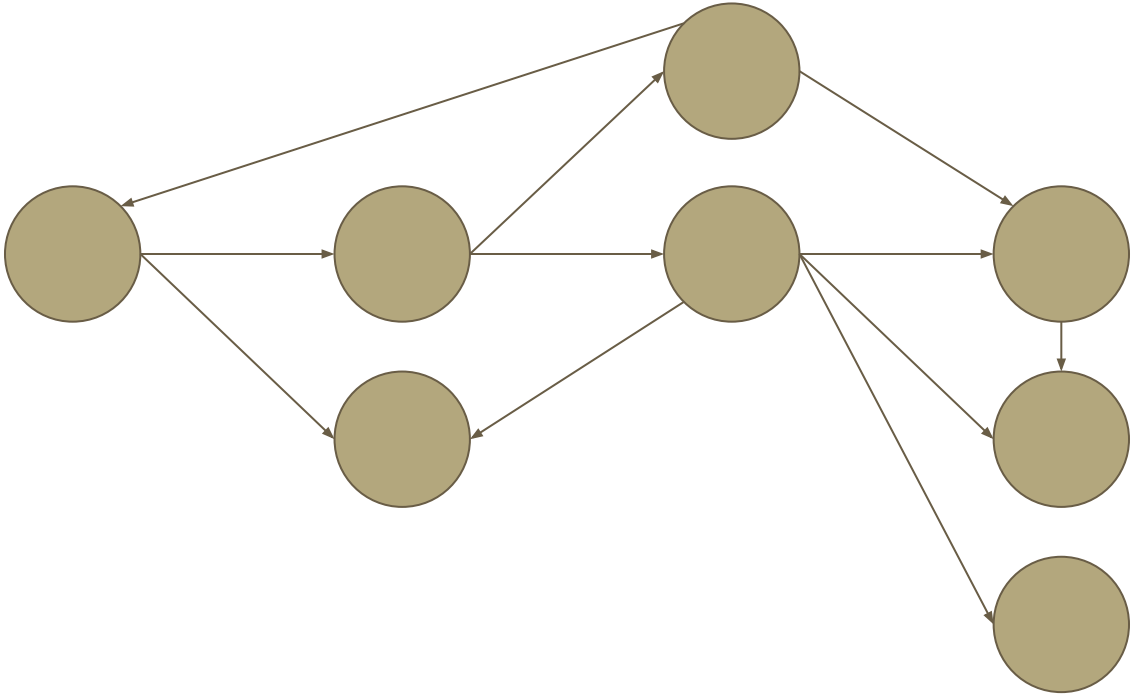
Lista



Árbol



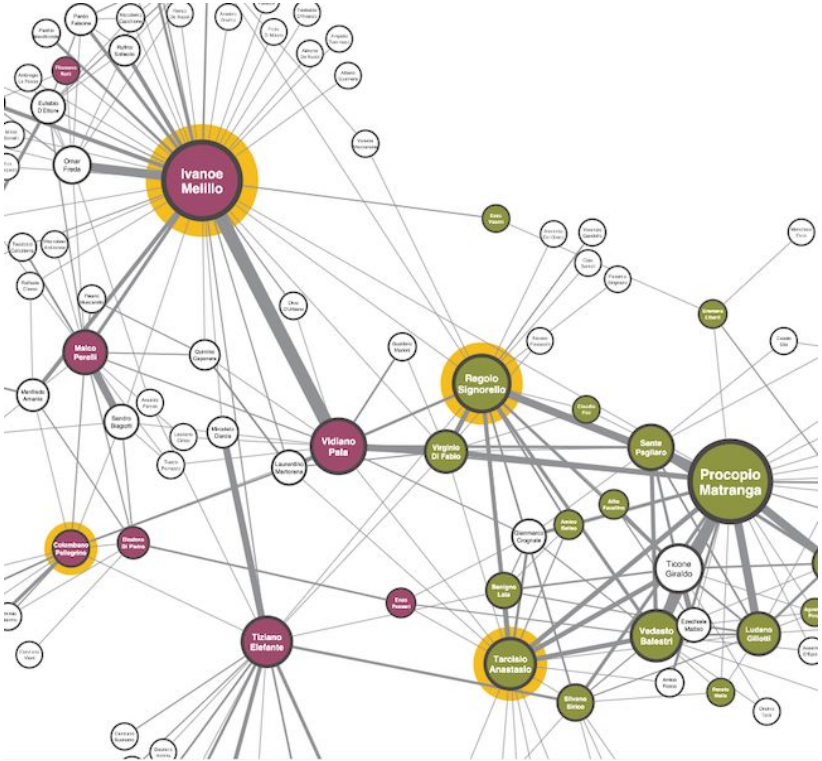
Grafo



Grafo



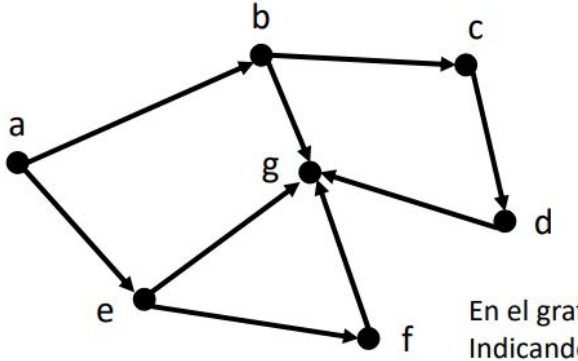
Grafo


Ivanoe Melillo
Melco Perelli
Vidiano Pala
Regolo Signorello
Virginio Di Fabio
Sante Pagliaro
Procopio Matranga
Colombano Pellegrino
Tiziano Elefante
Tarcisio Anastasio
Vadasio Balestri
Ludano Giliotti
Tirone Giraldo

Teoría de Grafos

Grafo Dirigido/Digrafo

Un grafo G=(V,E) es un conjunto V de vértices (o nodos) y un conjunto E de aristas dirigidas (VxV) o arcos dirigidos. Intuitivamente E permite representar una relación entre elementos de V.

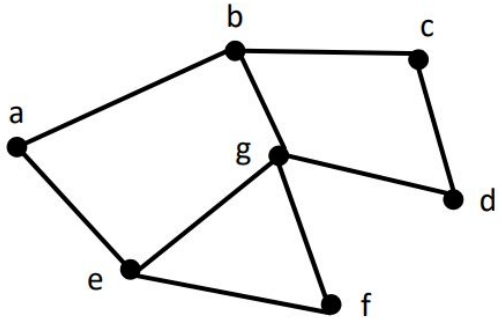


V = { a, b, c, d, e, f, g }
E = { (a,b), (b,c), (c,d), (d,g), (e,g), (e,f), (f,g), (a,e), (b,g) }

En el grafo dirigido, el arco (u,v) es un par ordenado
Indicando una flecha del vértice u al vértice v.

Grafo no Dirigido

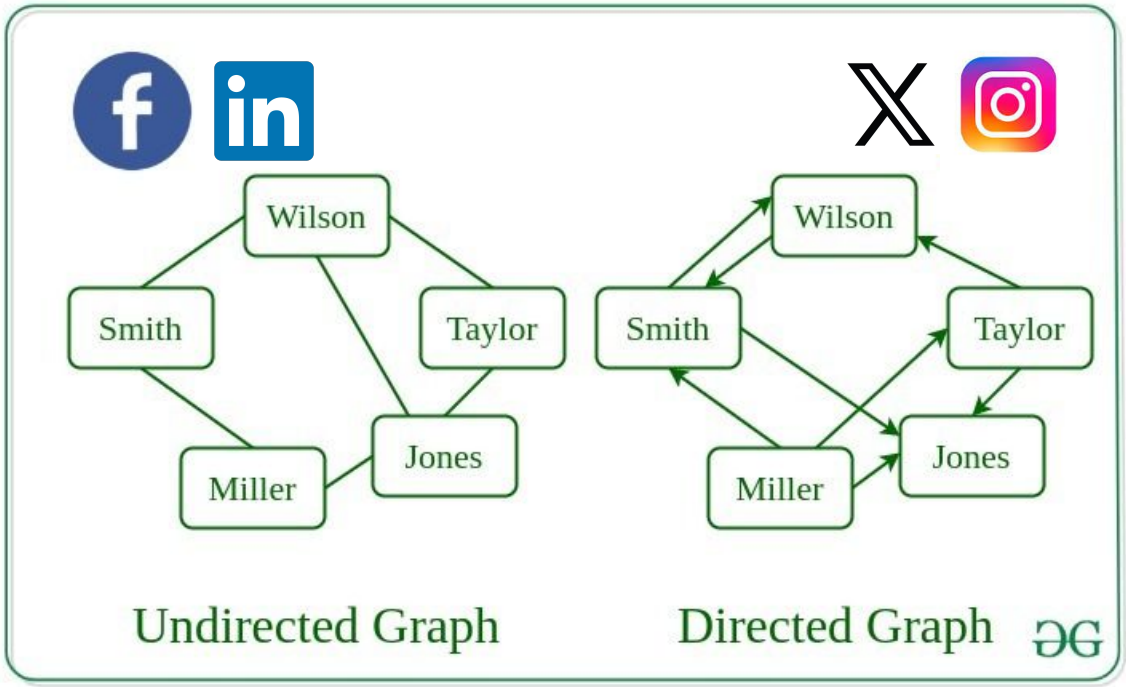
Un grafo G=(V,E) es un conjunto V de vértices (o nodos) y un conjunto E de aristas (VxV) o arcos. Intuitivamente E permite representar una relación (simétrica) entre elementos de V.



V = { a, b, c, d, e, f, g }
E = { (a,b), (b,c), (c,d), (d,g), (e,g), (e,f), (f,g), (a,e), (b,g) }

En el grafo no dirigido, el arco (u,v) en realidad es un conjunto {u,v} ya que (u,v) es lo mismo que (v,u).

Grafo no Dirigido / Grafo Dirigido



Arcos Emergentes

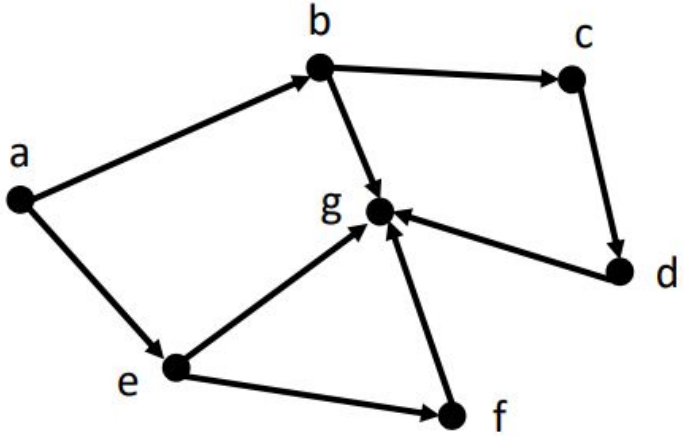
Se denomina “Arcos Emergentes” de un vértice *v* a todos los arcos que salen de *v*.

ArcosEmergentes(v) = { (v,y) : (v,y) pertenece a E }

Se denomina “Grado de Emergencia” de un vértice *v* a la cantidad de arcos que hay en el conjunto ArcosEmergentes(*v*).

Se denomina “Hojas del Digrafo” al conjunto de vértices del grafo cuyo grado de emergencia es 0.

Arcos Emergentes



ArcosEmergentes(b) =
{ (b,c), (b,g) }
Outdegree(b) = 2

El vértice g es una hoja del digrafo

Arcos Incidentes

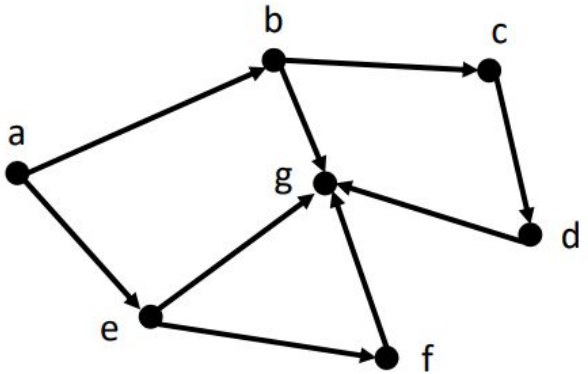
Se denomina “Arcos Incidentes” de un vértice *v* a todos los arcos que llegan a *v*.

ArcosIncidentes(v) = { (x,v) : (x,v) pertenece a E }

Se denomina “Grado de Incidencia” de un vértice *v* a la cantidad de arcos que hay en el conjunto ArcosIncidentes(*v*).

Se denomina “Raíces del Digrafo” al conjunto de vértices del grafo cuyo grado de incidencia es 0.

Arcos Incidentes



ArcosIncidentes(g) =
{ (b,g), (e,g), (d,g), (f,g) }
Indegree(g) = 4

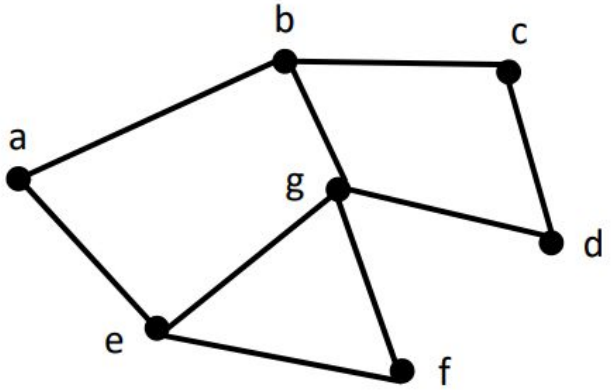
ArcosIncidentes(a) = { }
Indigree(a) = 0

El vértice a es una raíz del digrafo.

Grafo no Dirigido y Grado

En un grafo no dirigido, se habla de “grado” de un vértice porque los arcos que inciden y los que emergen son lo mismo.

Grado(v) = Cantidad de arcos que inciden en v



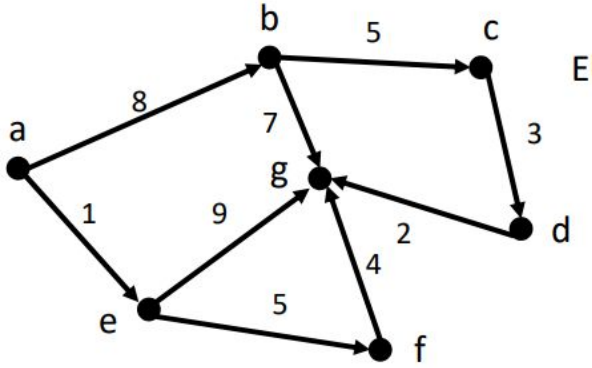
Grado(b) = 3
Grado(g) = 4

ArcosIncidentes(c) =
{(b,c), (c,d)}

Grafo Pesado

Un grafo pesado es un grafo donde las aristas contienen una etiqueta (o peso cuando las etiquetas son numéricas).

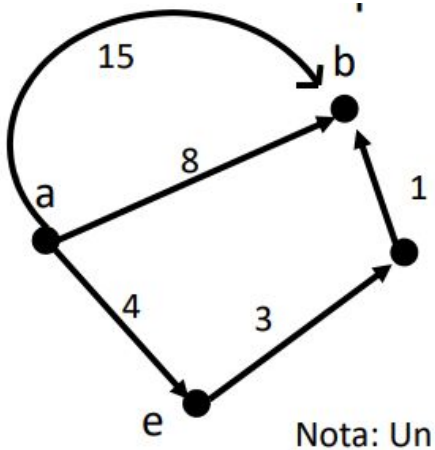
Ejemplo: Digrafo donde los vértices son aeropuertos y los arcos representan vuelos (o la distancia entre los aeropuertos)



El peso de (c,d) es 3.

Multi(di)grafo

Un multi(di)grafo es un (di)grafo donde un par de vértices puede estar conectado por una colección de arcos. Es decir, puede haber más de un arco entre cada par de vértices (se llaman arcos paralelos).

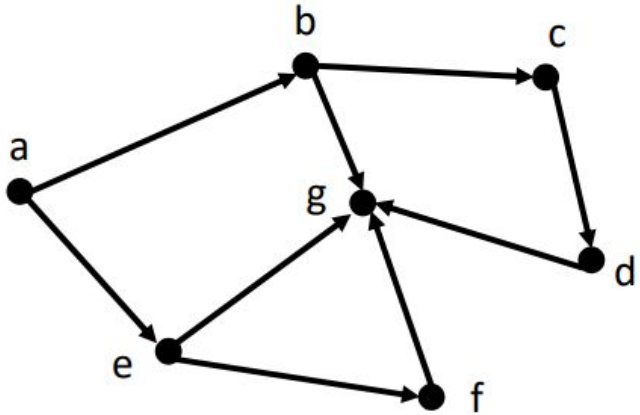


Hay dos arcos que unen “a” con “b”, uno con peso 8 y otro con peso 15.

Nota: Un “grafo simple” es un grafo que no es multigrafo.

Camino

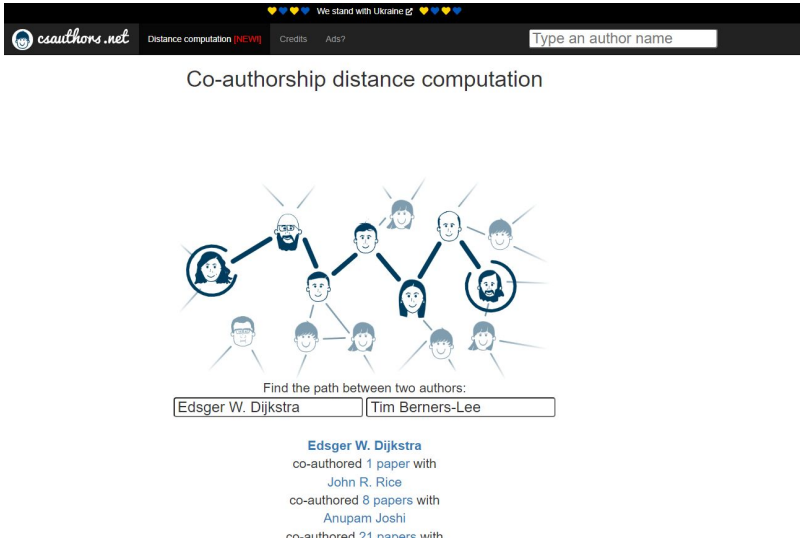
Un camino de v a w es una secuencia alternante de vértices y arcos tales que empieza en v y termina en w, y cada arco es incidente en sus vértices predecesor y sucesor.



a, b, c, d, g es un camino.
a, e, g es otro camino.

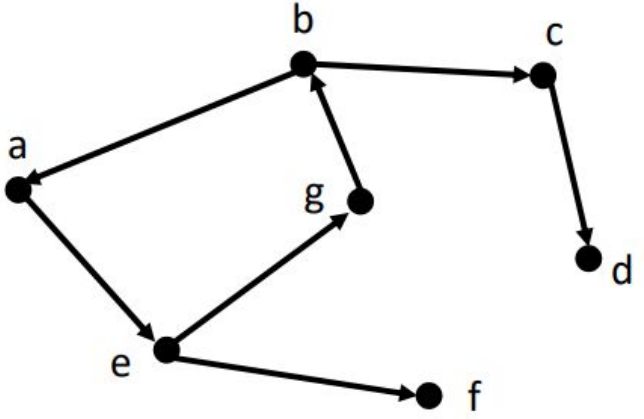
Camino / Número de Erdös

El número de Erdös de un investigador *X* es la longitud del camino de *X* a Paul Erdös en el grafo de coautores científicos. Paul Erdös fue el matemático más prolífico de la historia (publicó alrededor de 1500 artículos).



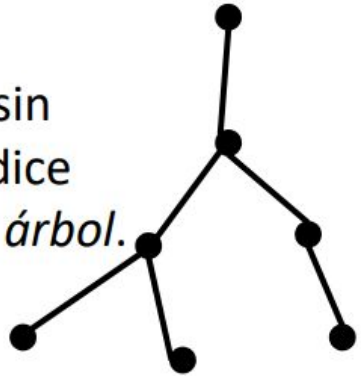
Ciclos

Un ciclo simple es un camino que comienza y termina en el mismo vértice sin repetir arcos. Un ciclo dirigido es un ciclo donde las aristas tienen dirección y son recorridas en su dirección.



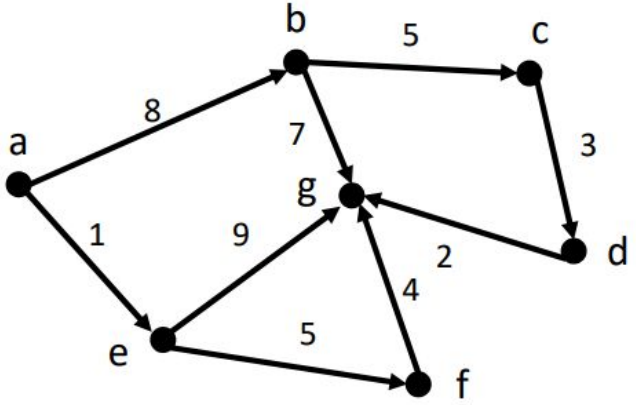
a, e, g, b, a es un ciclo dirigido.

Un grafo sin ciclos se dice acíclico o *árbol*.



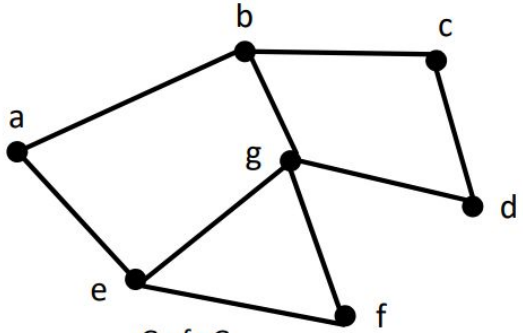
Ciclos

Costo de un camino (ciclo): Suma de los pesos de los arcos que forman el camino (ciclo)

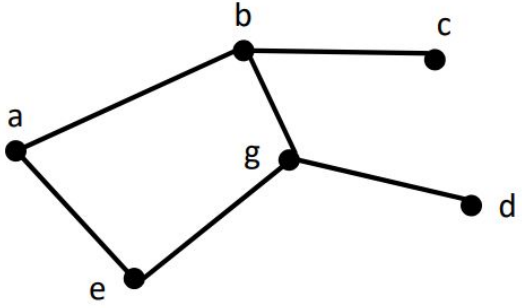


Subgrafos

Un subgrafo H de un grafo G es un grafo donde los vértices de H son un subconjunto de los vértices de G y los arcos de H son un subconjunto de los arcos de G.



Grafo G



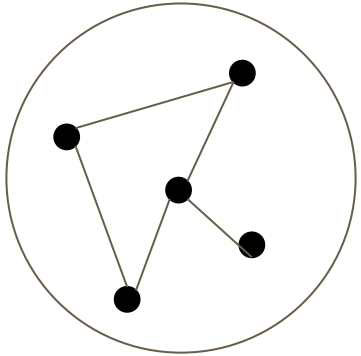
Grafo H subgrafo de G

Grafo Conexo

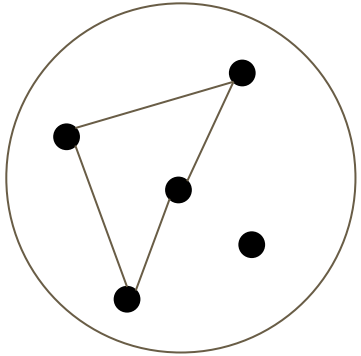
Un grafo G es conexo si para dos vértices cualquiera de G hay un camino entre ellos.

Si un grafo no es conexo, sus subgrafos maximales conexos se llaman componentes conexas.

Grafo conexo



Grafo no conexo



TDA Grafo no Dirigido

TDA Grafo no Dirigido

Graph<V,E>: Un grafo pesado de vértices con rótulos de tipo V y arcos con rótulos de tipo E

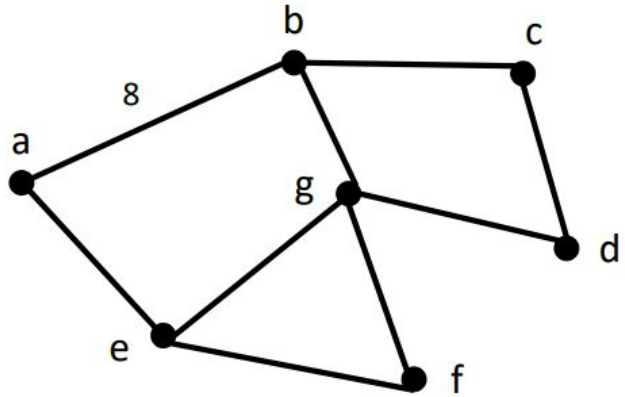
Vertex<V>: La posición de un vértice con rótulo de tipo V

Edge<E>: La posición de un arco con rótulo de tipo E

TDA Grafo no Dirigido

vertices(): Retorna una colección iterable con todos los vértices del grafo.

edges(): Retorna una colección iterable con todos los arcos del grafo.



G.vertices() → [va, vb, vc, vd, ve, vf, vg]

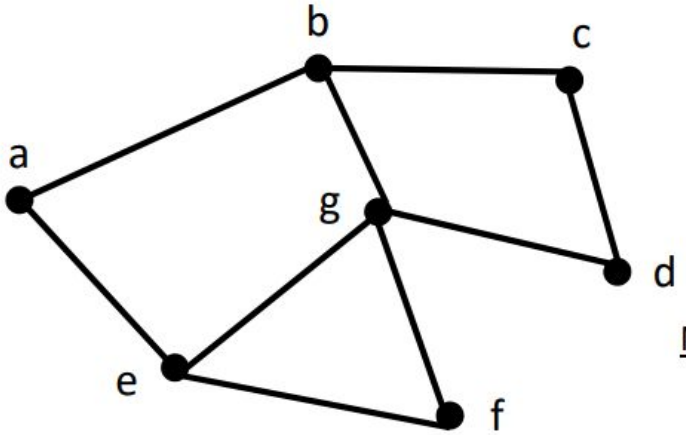
G.edges() → [e_{ab}, e_{ae}, e_{bc}, e_{bg}, e_{cd}, e_{dg}, e_{ef}, e_{eg}, e_{fg}]

va.element() → 'a'
e_{ab}.element() → 8

G.vertices().iterator().next().element() → 'a'
G.edges().iterator().next().element() → 8

TDA Grafo no Dirigido

incidentEdges(v): Retorna una colección iterable con todos los arcos incidentes sobre un vértice v



G.incidentEdges(vg) →
$[e_{bg}, e_{dg}, e_{eg}, e_{fg}]$

Notación: vg denota el vértice con rótulo g

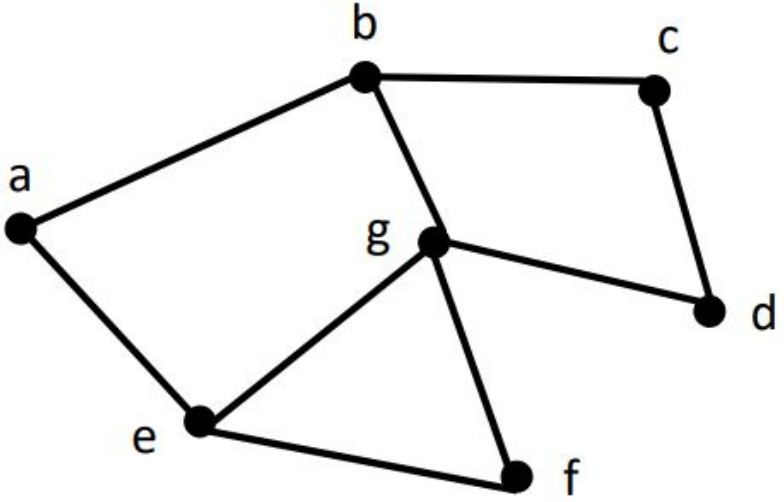
TDA Grafo no Dirigido

opposite(v,e): Retorna el otro vértice w del arco e=(v,w); ocurre un error si e no es incidente (o emergente de v).

endVertices(e): Retorna un arreglo (de 2 componentes) conteniendo los vértices del arco e.

areAdjacent(v,w): Testea si los vértices v y w son adyacentes.

TDA Grafo no Dirigido



G.opposite(vb, e_{bg})) → vg

G.endVertices(e_{bg}) →

vb	vg

G.areAdjacent(vb, vg) → true

G.areAdjacent(ve, vc) → false

TDA Grafo no Dirigido

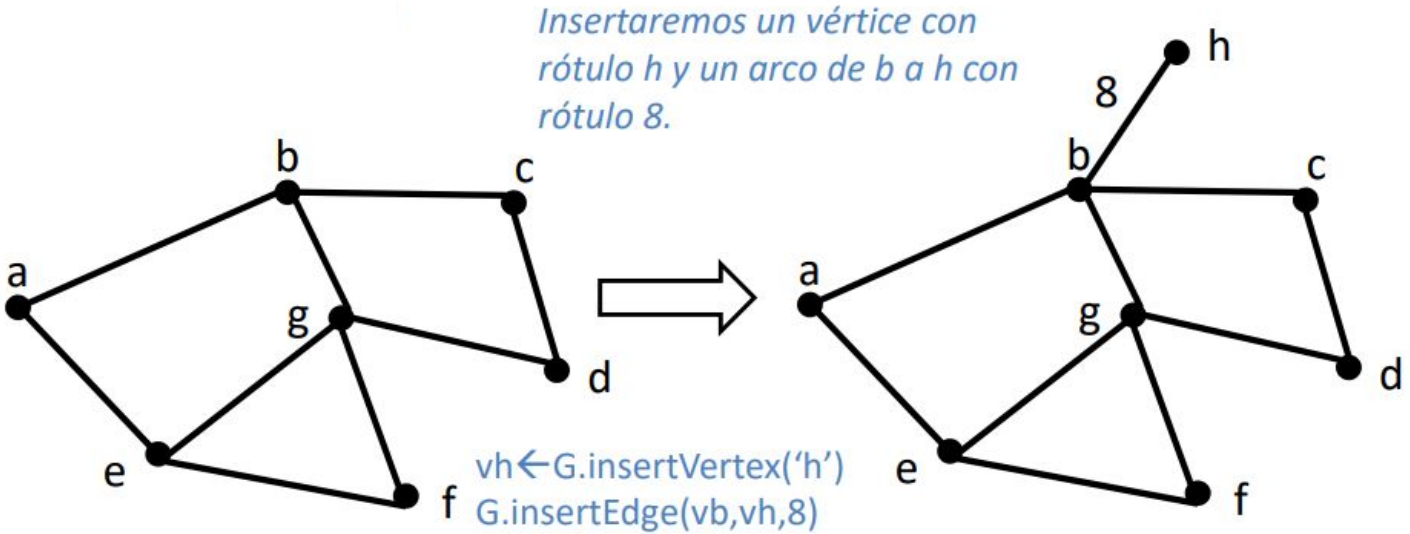
replace(v,x) : Reemplaza el rótulo del vértice v con x

replace(e,x): Reemplaza el rótulo del arco e con x

insertVertex(x): Inserta y retorna un nuevo vértice con rótulo x

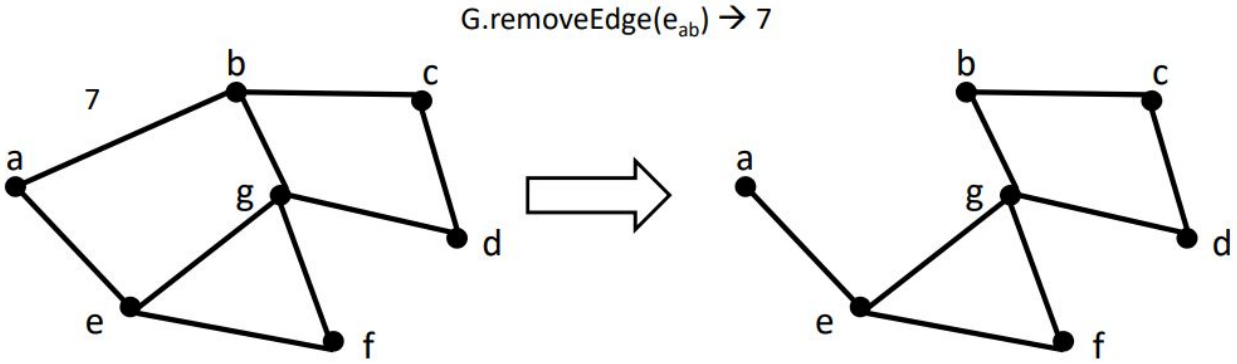
insertEdge(v, w, x): Inserta un arco con rótulo x entre los vértices v y w

TDA Grafo no Dirigido



TDA Grafo no Dirigido

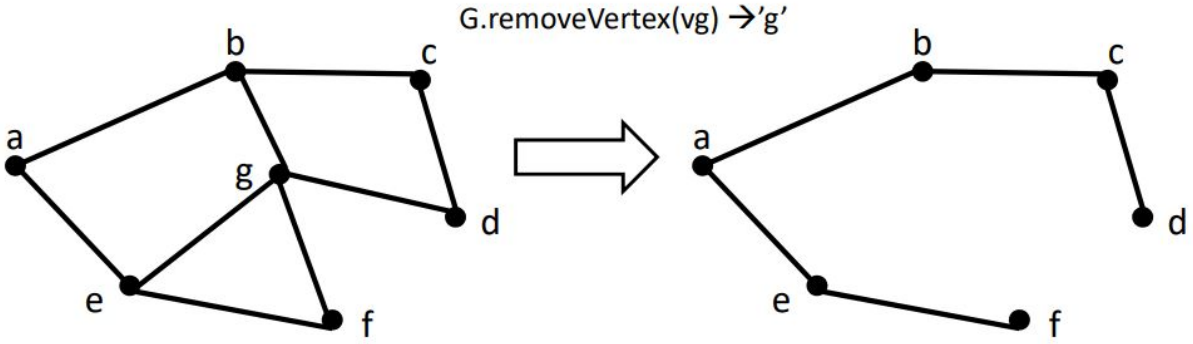
removeEdge(e): Elimina el arco e y retorna el rótulo almacenado en e.



Eliminamos el arco que conecta *a* con *b* con peso 7.

TDA Grafo no Dirigido

removeVertex(v): Elimina el vértice v y todos sus arcos adyacentes y retorna el rótulo de v



Break.

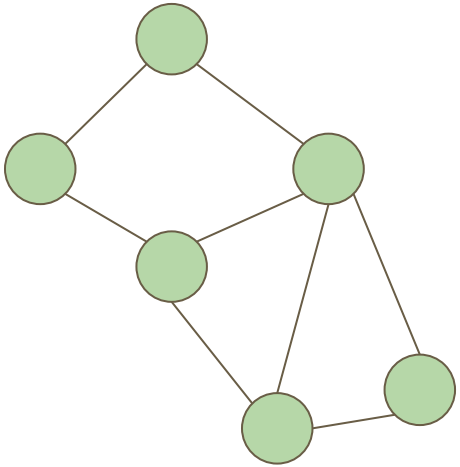
RECORRIDOS DE GRAFOS

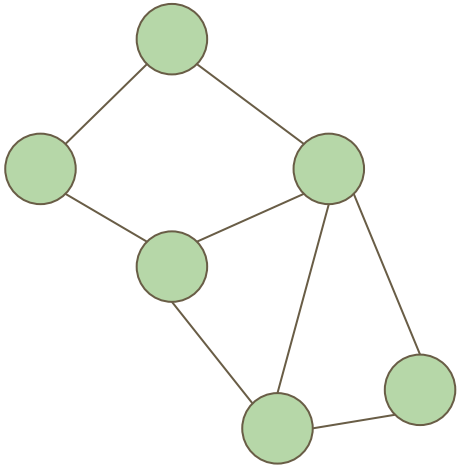
Recorridos de Grafos

En profundidad (Depth-First Search o DFS): (Equivale al recorrido pre o post orden en árboles + un testeo para no volver a recorrer un subgrafo ya explorado).

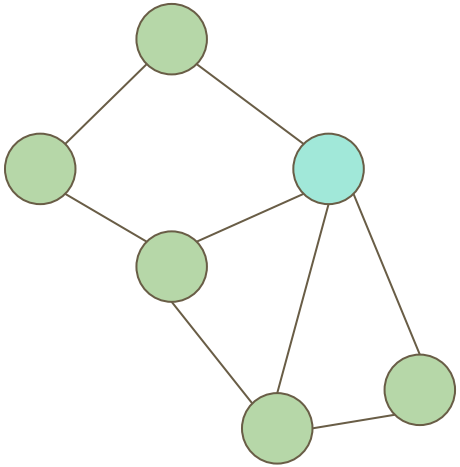
En anchura (Breadth-First Search o BFS): (Equivale al recorrido por niveles en árboles + un testeo para no volver a recorrer un subgrafo ya explorado).

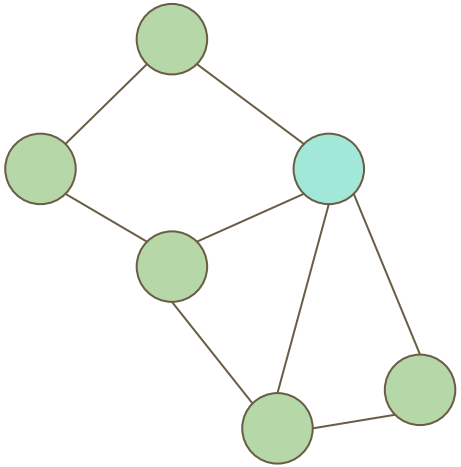
Recorridos de Grafos





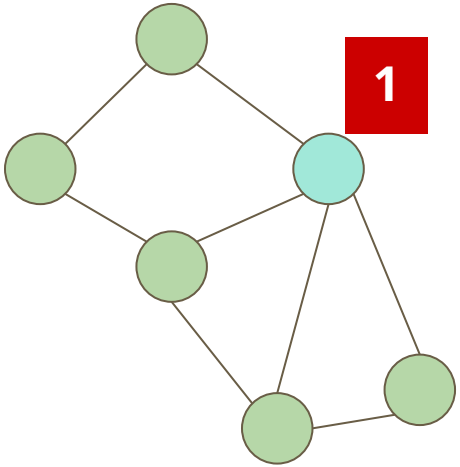
Recorridos de Grafos

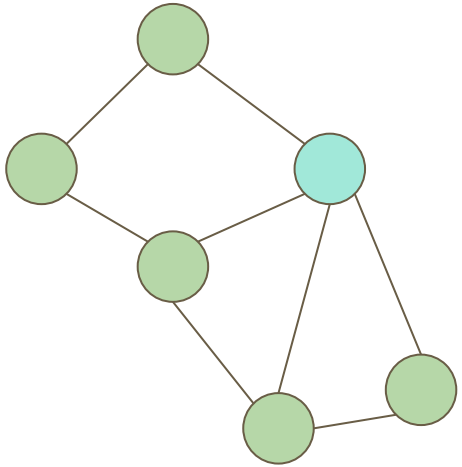




En Profundidad

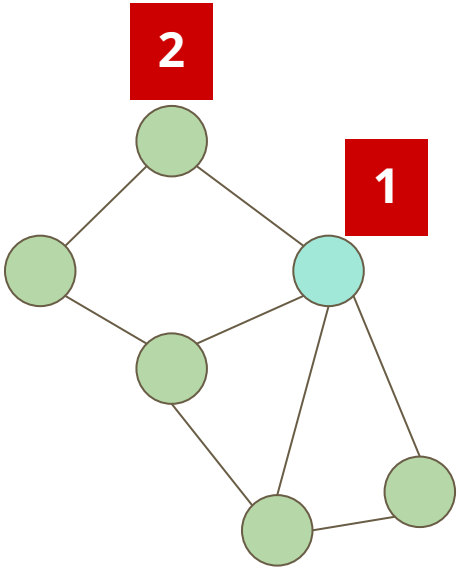
Recorridos de Grafos

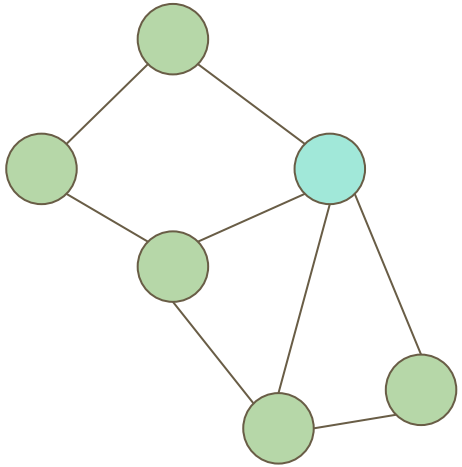




En Profundidad

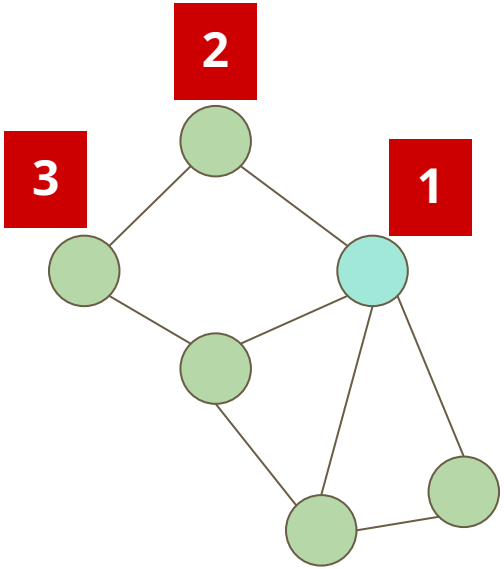
Recorridos de Grafos

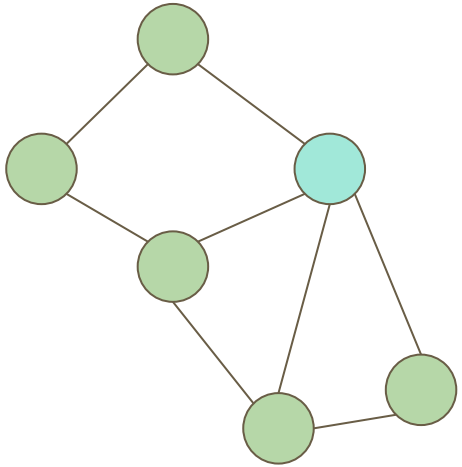




En Profundidad

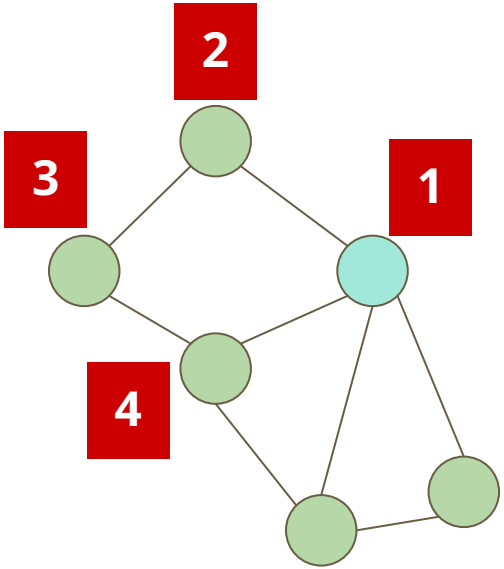
Recorridos de Grafos

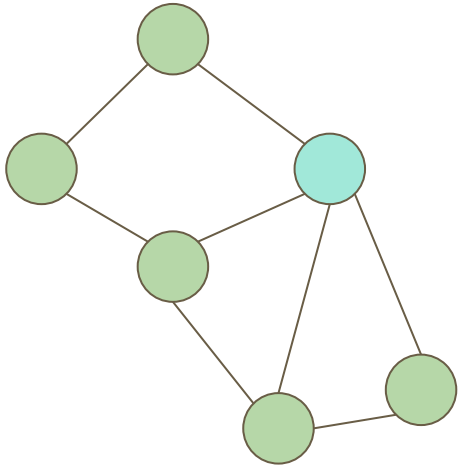




En Profundidad

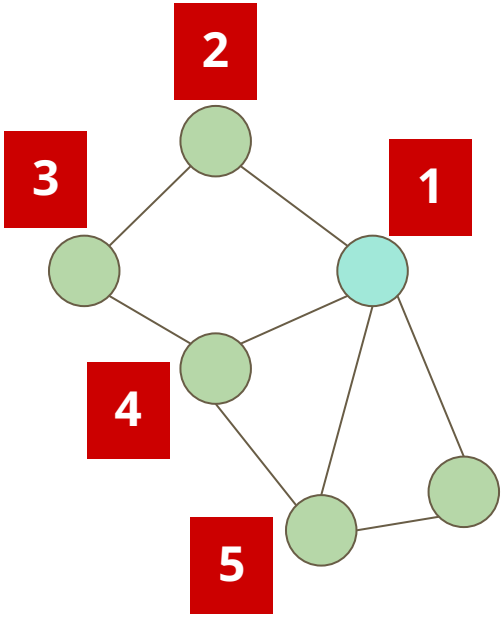
Recorridos de Grafos

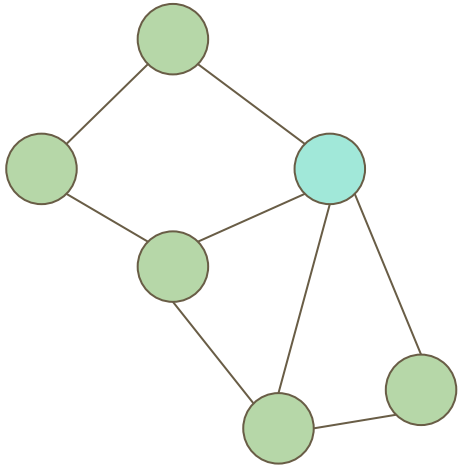




En Profundidad

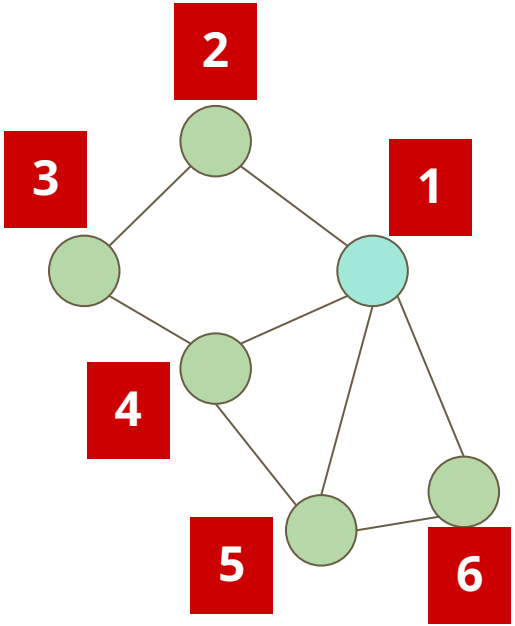
Recorridos de Grafos

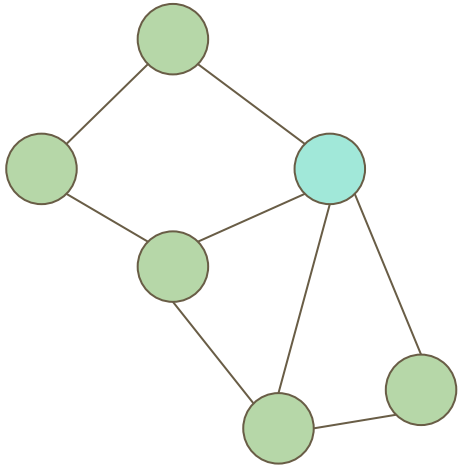




En Profundidad

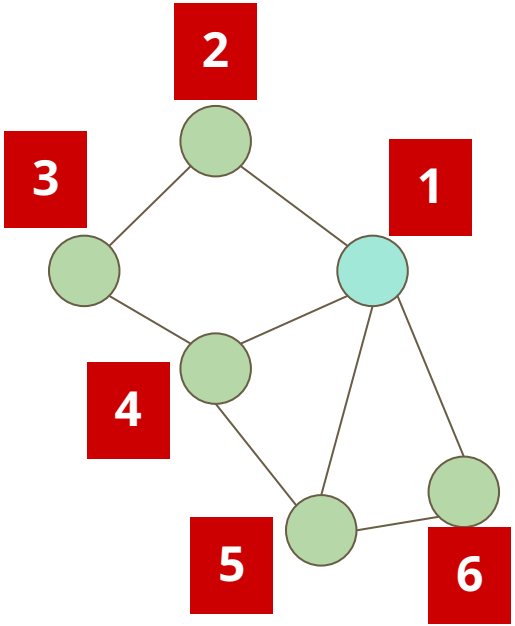
Recorridos de Grafos



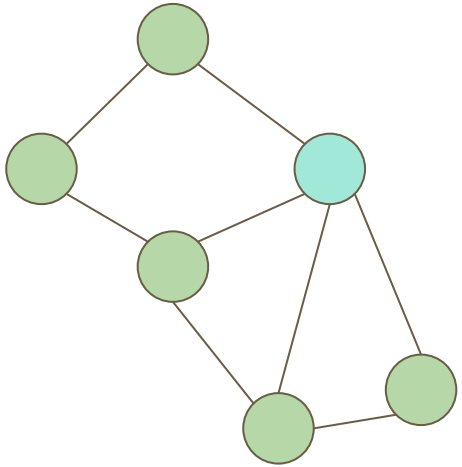


En Profundidad

Recorridos de Grafos

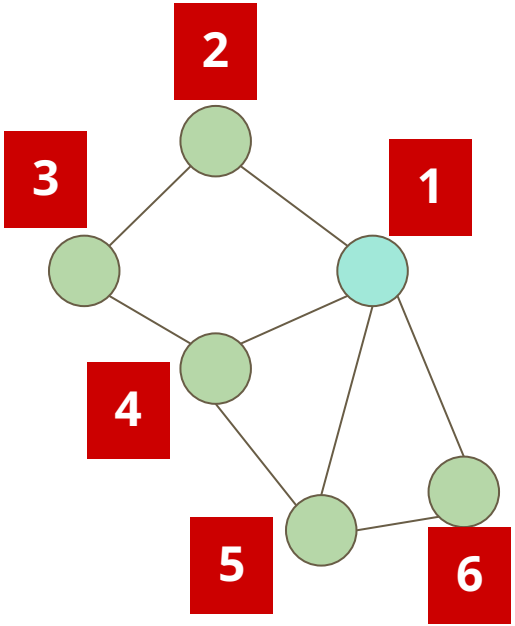


En Profundidad

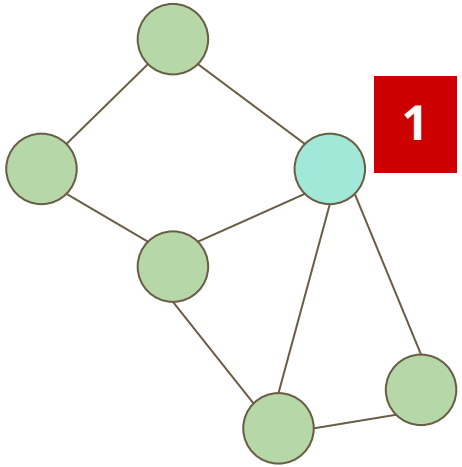


En Anchura

Recorridos de Grafos

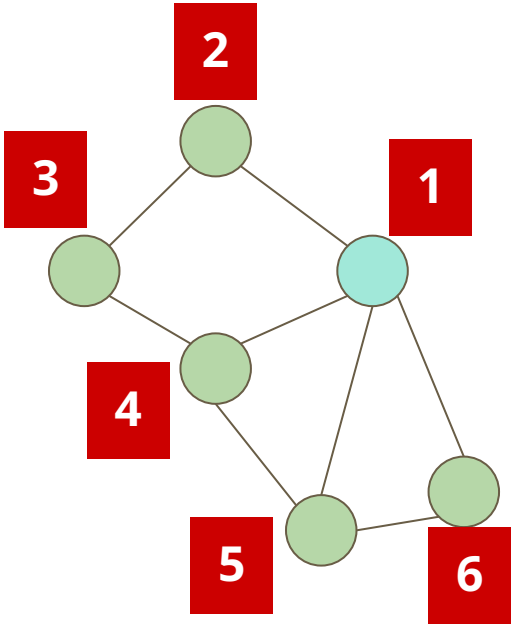


En Profundidad

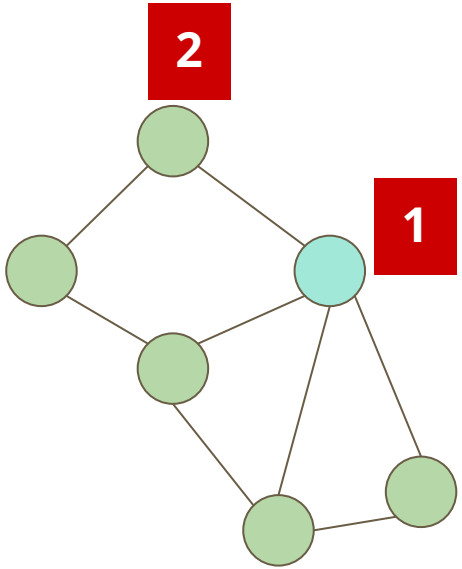


En Anchura

Recorridos de Grafos

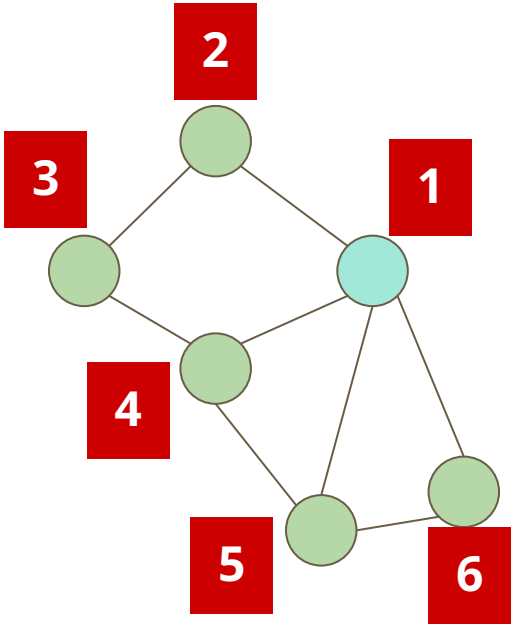


En Profundidad

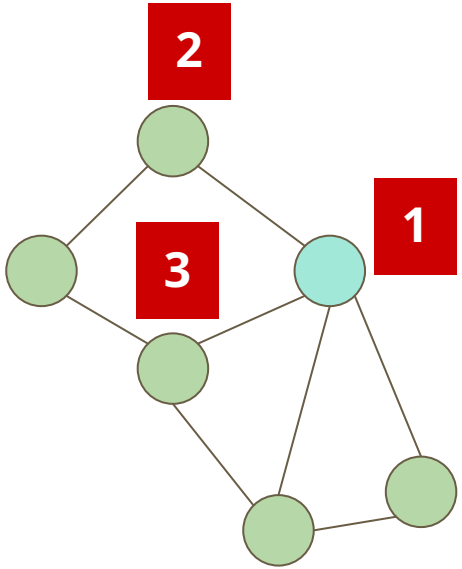


En Anchura

Recorridos de Grafos

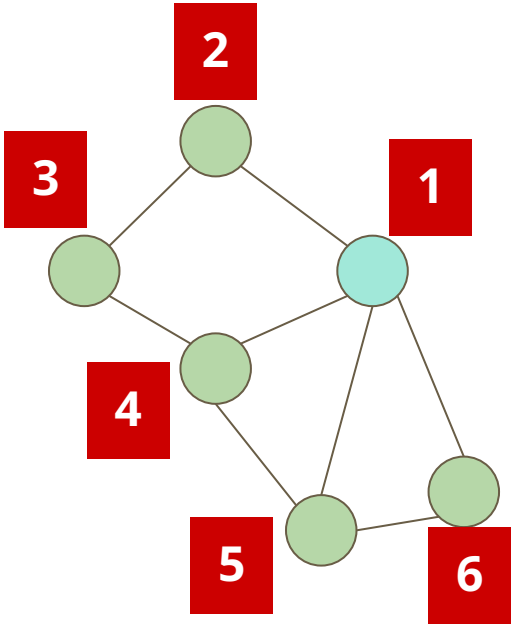


En Profundidad

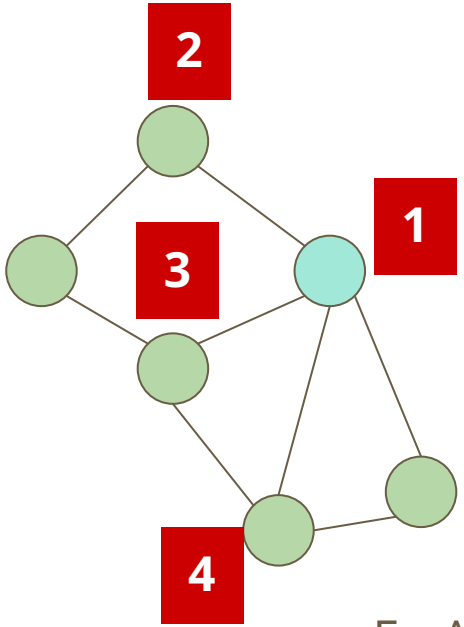


En Anchura

Recorridos de Grafos

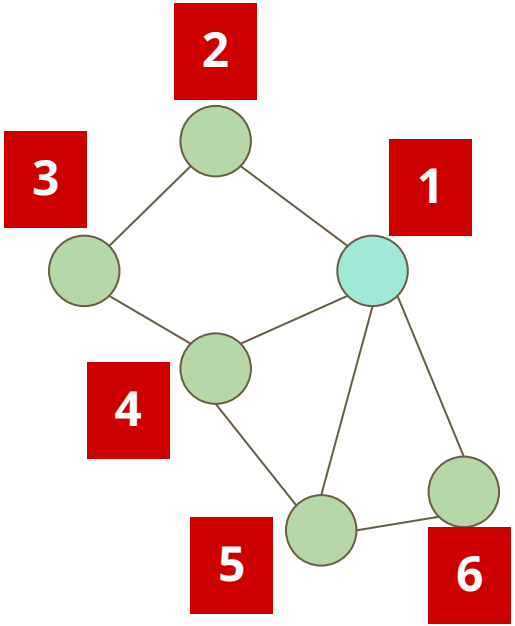


En Profundidad

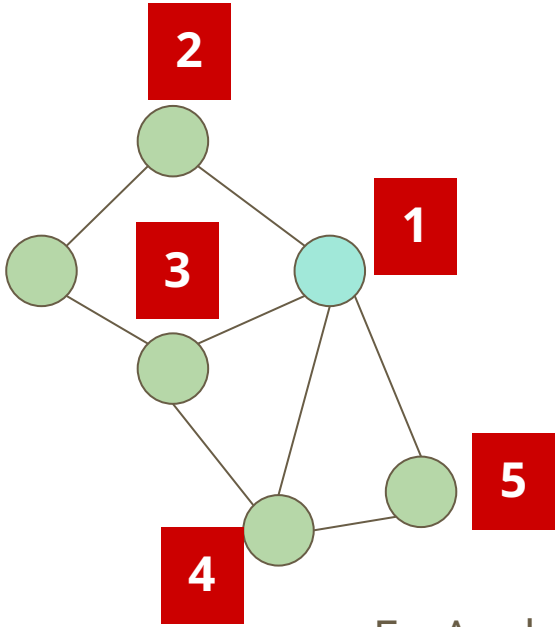


En Anchura

Recorridos de Grafos

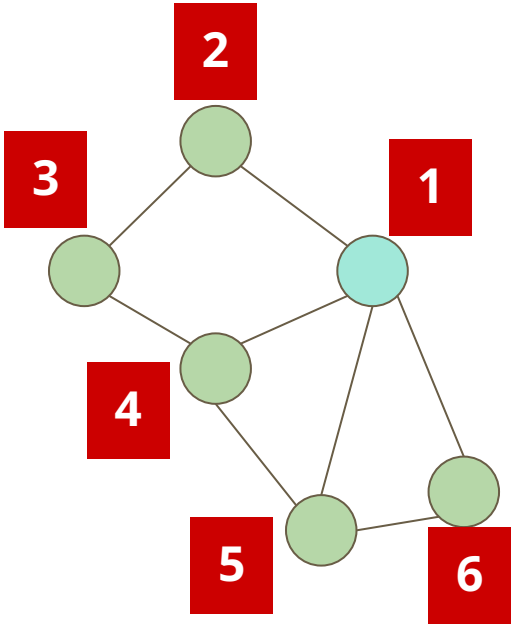


En Profundidad

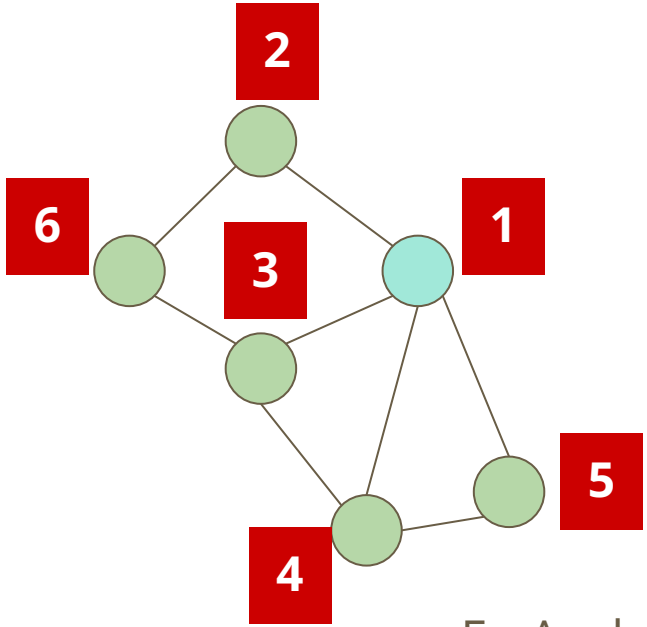


En Anchura

Recorridos de Grafos



En Profundidad



En Anchura

En Profundidad (DFS)

Una búsqueda en profundidad (DFS o Depth-First Search) permite recorrer todos los vértices de un grafo de manera ordenada.

Su funcionamiento consiste en ir expandiendo todos y cada uno de los nodos que va localizando, de forma recurrente, en un camino concreto.

Cuando ya no quedan más nodos que visitar en dicho camino, regresa (backtracking), de modo que repite el mismo proceso con cada uno de los hermanos del nodo ya procesado.

En Profundidad (DFS)

```
Algoritmo DFS( G : Grafo; v : Vértice )
    procesamiento de v previo al recorrido (visita pre)
    marcar a v como visitado
    para cada vértice w adyacente a v en G hacer
        si w no está visitado entonces DFS( G, w )
    procesamiento de v posterior al recorrido  (visita pos)
```

En Profundidad (DFS)

```
Algoritmo DFSShell( G : Grafo )
    para cada vértice v de G hacer
        marcar v como no visitado
    para cada vértice v de G hacer
        si v no está visitado entonces
            DFS( G, v )
```

$T(n,m)=O(n+m)$

¿Cómo hacer el “visitado”?

1. Usar un mapeo externo al grafo de Vertex en Boolean

    – Ventaja: No hay que modificar el grafo que ya programamos.

    – Contra: El tiempo de marcar y desmarcar puede tender a O(n)

2. Agregar un boolean a la clase Vertice del grafo

    – Ventaja: Puedo garantizar que marcar y desmarcar funciona en O(1)

    – Contra: Por cada algoritmo que escribo tengo que ensuciar el grafo agregando atributos y operaciones, hay que modificar la interface

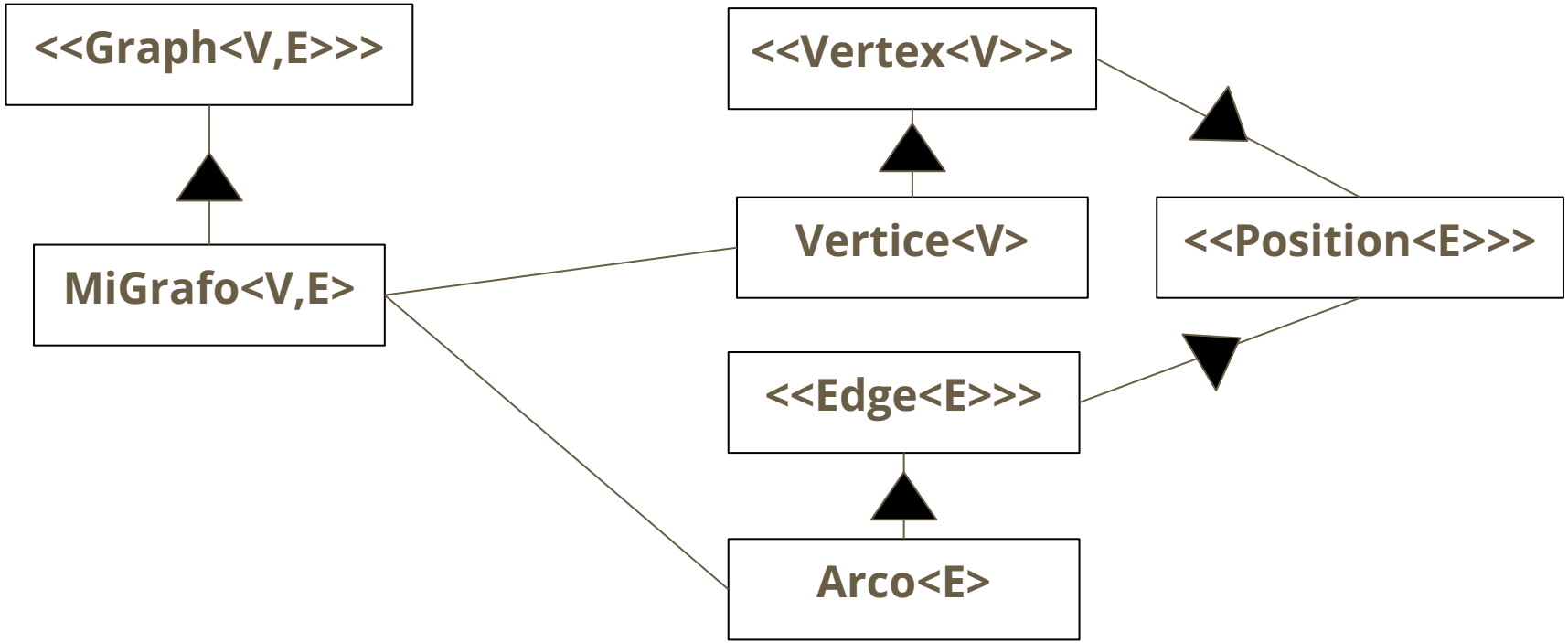
¿Cómo hacer el “visitado”?

3. Decorar los vértices del grafo (opción de GT).

– Ventaja: En forma abstracta mantengo toda la información del DFS y de futuros algoritmos

– Contra: Hay que modificar lo que ya programamos

Spoiler sobre implementación



Decorar los vértices

Una posición decorada es una posición que además es un mapeo.

```
public interface DecorablePosition<E> extends Position<E>, Map<Object,
Object> {

… sin métodos nuevos… }

public interface Vertex<V> extends DecorablePosition<V> { }

public interface Edge<E> extends DecorablePosition<E> { }
```

Decorar los vértices

Lo retomaremos en la siguiente clase cuando veamos la implementación :)

Bosques del DFS en Grafos no Dirigidos

Las llamadas sucesivas a DFS desde DFSShell van descubriendo vértices que forman árboles.

Estos árboles forman el bosque (o foresta) del DFS. Al orientar los arcos en la dirección en la que son explorados durante el recorrido, se distinguen:

– Arcos de descubrimiento o árbol (discovery o tree edges): Arcos que llevan a vértices no visitados

– Arcos de retroceso (back edges): Arcos que llevan a vértices que ya fueron visitados.

Usos de DFS en Grafos no Dirigidos

Testear si G es conexo (todos los vértices quedan visitados si y sólo si el grafo es conexo)

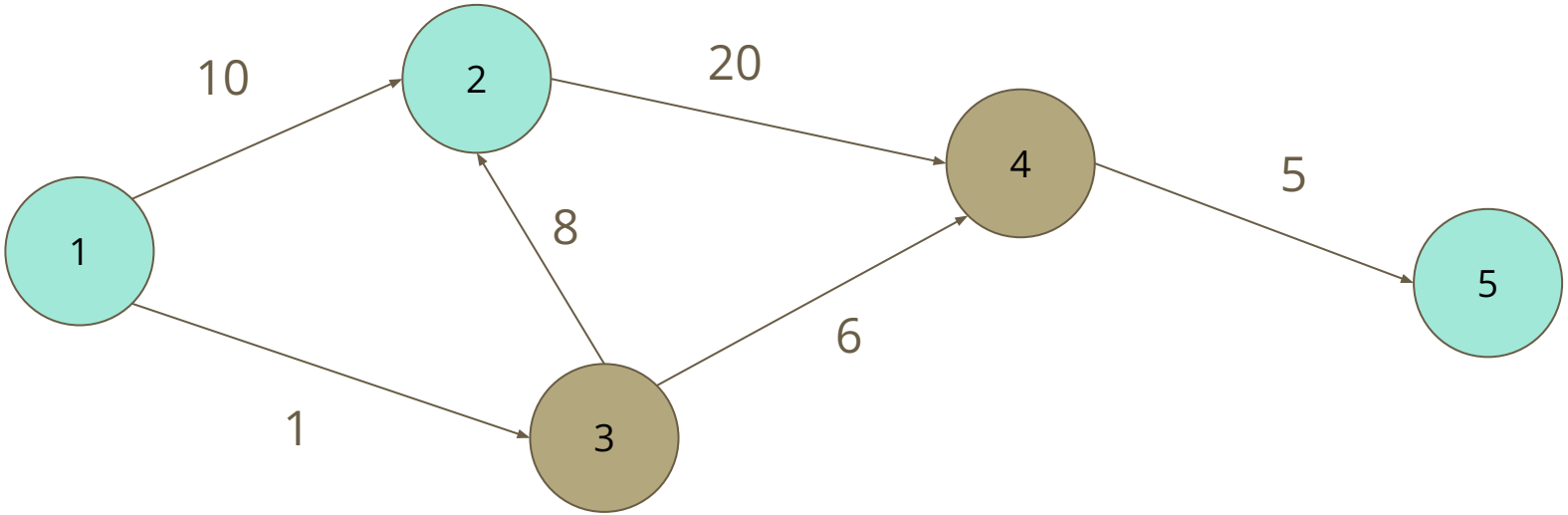
Calcular las componentes conexas (por cada iteración de DFSShell incremento un contador indicando el número de componente conexa y con ese contador etiqueto los vértices de cada componente)

Encontrar un camino entre dos nodos

Encontrar un ciclo

Encontrar el camino de menor costo entre dos nodos (costo mínimo)

Estrategia para buscar camino de costo mínimo



Estrategia para buscar camino de costo mínimo

La estrategia consiste de generar por fuerza bruta todos los caminos posibles de origen a destino con su costo. Hago un proceso de mínimo: Cada vez que encuentro un nuevo camino de origen a destino, me fijo si el costo del nuevo camino hallado es mejor que el mejor que tengo hasta este momento. Si es así, lo actualizo. Para generar todos los caminos posibles debo:

(1) Marcar vértices como visitados para no entrar en bucles infinitos si el grafo tiene ciclos dirigidos.

(2) Desmarcar vértices al hacer backtracking para permitir que un vértice pueda pertenecer a más de un camino posible.

Búsqueda en Anchura, BFS

La búsqueda en anchura (BFS o Breadth First Search) es un algoritmo para recorrer o buscar elementos en un grafo.

Se comienza eligiendo algún nodo como elemento raíz y se exploran todos los vecinos de este nodo.

A continuación para cada uno de los vecinos se exploran sus respectivos vecinos adyacentes, y así hasta que se recorra todo el grafo.

Búsqueda en Anchura, BFS

```
Algoritmo BFS( G : Grafo; v : Vértice )
        cola ← new Cola()
        cola.enqueue( v )
        marcar a v como visitado
        mientras not cola.isEmpty() hacer
                u  ← cola.dequeue()
                procesar a u
                para cada vértice x adyacente a u hacer
                        si x no está visitado entonces
                                marcar a x como visitado
                                cola.enqueue( x )
```

Búsqueda en Anchura, BFS

```
Algoritmo BFSShell( G : Grafo )
    para cada vértice v de G hacer
        marcar v como no visitado
    para cada vértice v de G hacer
        si v no está visitado entonces
            BFS( G, v )
```

$T(n,m)=O(n+m)$

Búsqueda en Anchura, BFS - Aplicaciones

Hallar el camino más corto (en cantidad de arcos) entre dos vértices (en $O(n+m)$)

Bibliografía

Capítulo 13 de M. Goodrich & R. Tamassia, Data Structures and Algorithms in Java. Fourth Edition, John Wiley & Sons, 2006.

Capítulo 9 de M.A.Weiss, Data Structures and Algorithm Analysis in Java, Third Edition, Pearson, 2012.

THE END.